

Java

VS

Java

Profiling Different Versions of Java
& Different methods in Java

Gurodishtzer Aaron
Hadad Eyal

Table of Contents

Overview.....	- 3 -
Java In Details.....	- 4 -
Hardware details:.....	- 5 -
Ubuntu Setup.....	- 6 -
Copy An Array.....	- 7 -
Windows 10, Local Machine – JAVA 8	- 8 -
Windows 10, Local Machine – JAVA 11	- 8 -
Windows 10, Local Machine – JAVA 17	- 9 -
Conclusions:	- 10 -
Ubuntu18.04 VS Ubuntu22.04.....	- 11 -
Conclusions:	- 12 -
Java Streams – Powerful but which way ?	- 14 -
Local Machine – Ubuntu:	- 15 -
Record vs. Class	- 17 -
Ubuntu18.04	- 17 -
Ubuntu22.04	- 17 -
What should I do Switch-Case or If and If else ?	- 19 -
Ubuntu 18.04	- 19 -
Ubuntu 22.04	- 19 -
Using CONSTANTS – Enum or Static final ?	- 21 -
Ubuntu 18.04	- 21 -
Ubuntu 22.04	- 23 -
Conclusions	- 23 -
Should I use a Stream or a For loop or ForEach ?.....	- 24 -
Ubuntu18.04 – Remote Machine – EC2:.....	- 24 -
Ubuntu22.04 – Remote Machine – EC2:.....	- 27 -
Conclusion.....	- 30 -
Different way to Read \ Write Files.....	- 31 -
Ubuntu 18.04	- 31 -
Ubuntu 22.04	- 32 -
Conclusions	- 33 -
Conclusions.....	- 34 -
Features in each version	- 35 -
References	- 36 -

Overview

In this modern world there are various ways to do the exact same thing, you can use programming languages to develop different applications, algorithms, etc.

While researching on the subject we came across a few researches which did a benchmarking test between Java and C language, and it is not hard to guess which will perform better but still, it is curious to find by how much, also since Kotlin came to replace Java at Android app development we also found a comparison between these two, and this is interesting since one came to replace another, the result from the paper were – “ The differences are in fact usually not very substantial, however, a common trend of Kotlin always being out-performed by Java is observed, even if that is not by a very big factor in the vast majority of benchmarks “[\[4\]](#)

When comparing two different programming languages, you will choose the best and most efficient way to implement the program in each language, and the result will decide which programming language will win in this contest.

But how you will know you chose the best approach to implement this ?

We are trying to figure out what is the best approach to do something since there are various ways of doing it.

Over the years since Java was released in May 1995, a lot of changes and features has been adding to it, stays the same in the aspect of high-level and object-oriented and run on the Java Virtual Machine (JVM).

There are currently 18 Java versions and, a new version is scheduled for every 6 months [\[2\]](#), we decided to start our testing from Java 8 since legacy project companies still using Java 8 or actually stuck with it.

There are 3 Long Term Support (LTS) Java versions, and the main focus will be on them.

- Java 8, 11, 17.

Each version come with a set of features some are an addition to tool set and some come to replace existing methods, we list some of the features but definitely not all and we are do not intend to, since it is out of scope for this paper.

You can read about the new features that each version is adding to the Java language, since the JDK is a superset of the JRE that have inside it the JRE some versions mostly updating the JVM and the garbage collector (GC), we will not cover these changes.

We will check the performance of “*doing the same thing just in a different way*”, on the different versions of Java. Also check the same code on different versions of Java Development Kit (JDK) and Java Runtime Environment (JRE).

JRE:

It is a package of everything necessary to run a **compiled** Java program, including the Java Virtual Machine (JVM), the Java Class Library, the *java* command, and other infrastructure.

However, it cannot be used to create new programs.

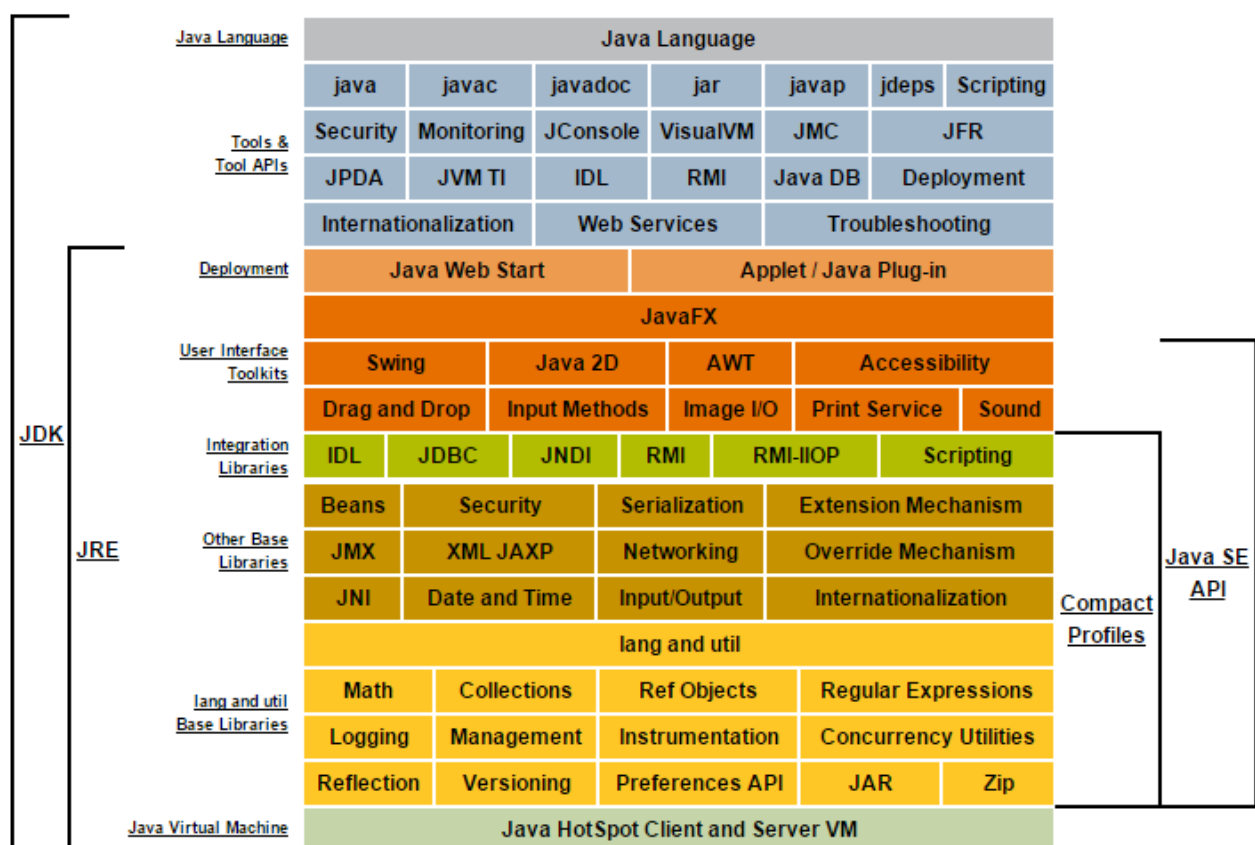
JDK:

The JDK is the full-featured SDK (Software Development Kit) for Java.

It has everything the JRE has, but also the compiler (*javac*) and tools (like *javadoc* and *jdb*).

It is capable of creating and compiling programs.

Description of Java Conceptual Diagram



Hardware details:

Most servers run on Ubuntu Linux, and not always have the latest version of the OS, we decided to run these tests also on two different machines, since legacy project companies still running on Java 8, it is making sense that there is server out there that most likely do not use the latest version, and even the one before that.

We will run on the Amazon Web Services (AWS) and use instances from the Elastic Compute Cloud (EC2), here is a quote from their website that should explain some:

*"Amazon Elastic Compute Cloud (Amazon EC2) provides scalable computing capacity in the Amazon Web Services (AWS) Cloud.
Using Amazon EC2 eliminates your need to invest in hardware up front, so you can develop and deploy applications faster.
You can use Amazon EC2 to launch as many or as few virtual servers as you need, configure security and networking, and manage storage.
Amazon EC2 enables you to scale up or down to handle changes in requirements or spikes in popularity, reducing your need to forecast traffic."* [\[5\]](#)

In this project we use 2 EC2 instances, one run Ubuntu 22.04 and the other Ubuntu 18.04 this difference is also a parameter we are comparing since there servers that still using Ubuntu 18.04 version.

We connected to these instances to run a jar file that contains the code that we would run per case that we are observing and using YourKit Java profiler in order to monitor the JVM in the remote instance.

We will run on two different hardware, the AWS EC2 hardware and our local machine Hardware – it has big gap in difference.

Local Machine:

Processor: 11th Gen Intel® Core™ i7-11800, [8-cores]
16.0 GB RAM, (15.7 GB usable)

AWS EC2 Instance:

Processor: t2.micro [1-core]
1 GB RAM, (974 MB usable)

Since the EC2 instance has only 1 – core it will affect its performance, this will be perfect for our comparing since we should really feel the difference when using difference JDK since there are big upgrade involving the GC (Garbage Collector), needless to say, servers definitely have better hardware than the EC2 instance, and for that reason only we are running on our local machine.

Ubuntu Setup

For each machine before running any Java program we had to do minor changes.
install JDK for each version and also, we needed to find a way to quickly switch between versions.

After the machine is up & running do this steps.

1. *sudo apt update && sudo apt upgrade -y*

The -y flag just so you will not need to press Y when there is a confirmation of update that need to be install. But these updates are necessary in order to install any JDK version in both OS version.

Once finished updating. Installing the JDK versions. Order is irrelevant.

2. *sudo apt install openjdk-<>-jdk-headless*

In the diamond brackets need to specify the number of version you want to install we install only 8,11,17 for this research.

Since we will not use ant GUI in our performance testing, we install the headless versions.

3. need to find a way to switch quickly between versions.

sudo update-alternatives --config java

this command doses the trick very efficiently. Just read the instructions and you are good to go.

Copy An Array

Usually when you written probably you use an array as a simple data structure, and once in a while you need to work on the data with a non-destructive approach, hence you will copy the data.

There are various ways to copy an array in Java, we chose the most common ways.

1. for loop – the intuitive way
2. Using the '`java.lang.Object.clone()`'
3. Using `Arrays.copyOf()` - '`java.util`'
4. Using '`java.lang.System.arraycopy()`'
5. for each loop – mostly used on Data Structures

~ Note

We ran each method with a different class and one main thread that calling each method at a time, to compare the usage of the CPU respectively.

The order is as mentioned above, it matters ! look at Heap Image

Starting with *Windows 10* example just to get a perspective of the *copy-Array* performance
Than we will run on our remote machine to see the difference when we have only one core and 1-GB RAM.

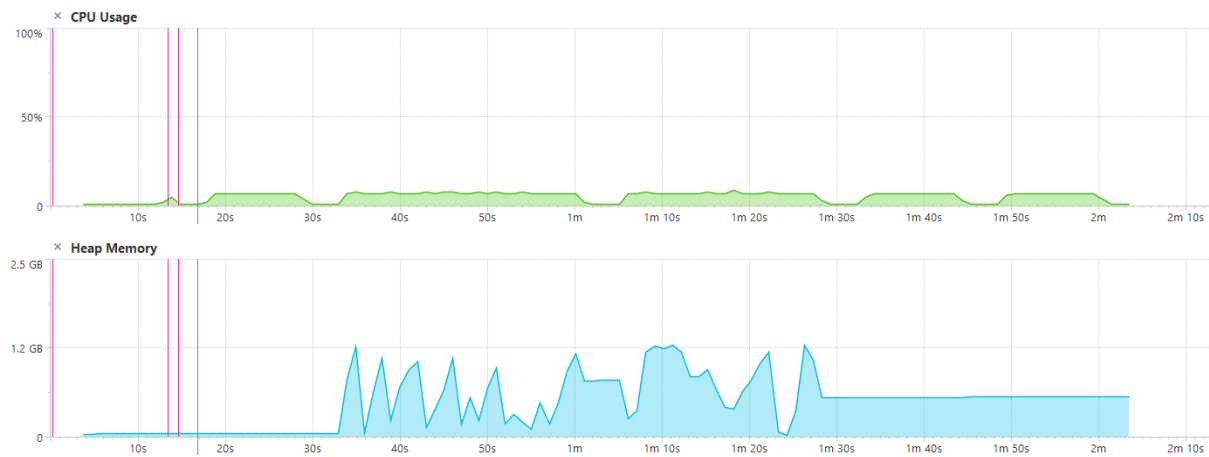
Note.

This note is for all performance tests we will run on the remote machine – since there is only one core, when attaching the profiler to the Java program we will see a lot of the time differences since there will be a lot of *CONTEX-SWITCHING* in order for the CPU to give each Thread the required time. We will take that parameter in count so when reproducing our result please take that in mind.

Steps to reproduce:

1. create 16 arrays two for each primitive type SIZE = 100,000:
byte, short, int, long, float, double, char, boolean
2. fill 8 arrays, each one with a supported value, randomness is redundant it can be the same value for each cell of the array.
3. Create a for-loop that will run 100,000 inside a separate class that will have access to the 16 arrays mentioned in step (1)
4. In each iteration Copy each full array to an empty one – after the first iteration both arrays will be full but that doesn't matter.
5. You should have by now 5 classes each one with a for-loop inside that for loop 8 arrays that being copied
6. We run everything from 1 Main Thread, 5 seconds sleep between each class.

Windows 10, Local Machine – JAVA 8



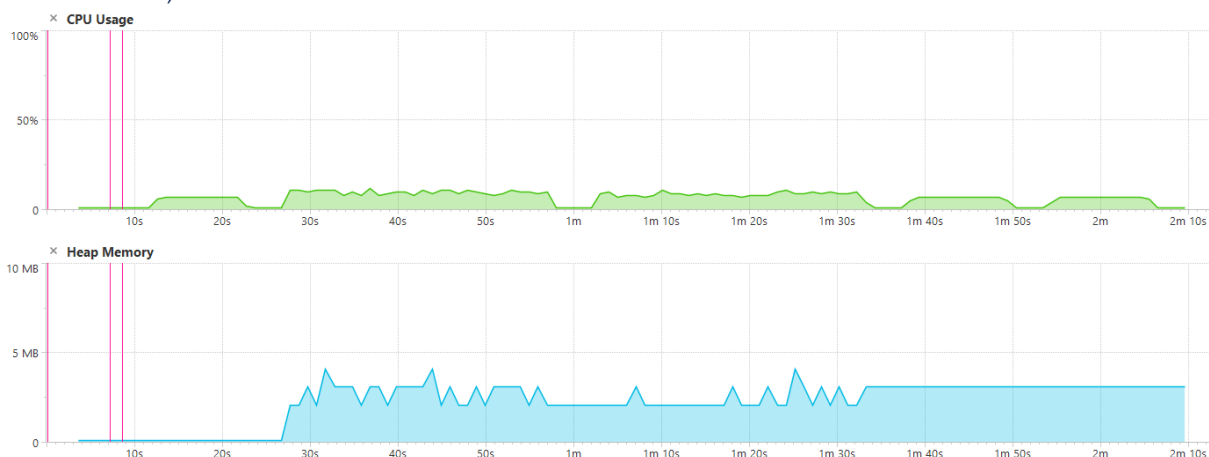
We can see the Heap-memory usage when reaching to 2,3 methods, no usage at all for the other methods (1,4,5), It explains the CPU usage results (of the Main thread).

~when changing the order of method 2 and 3 the Heap usage is similar.

Call Tree		Time (ms)	
main Group: 'main'		80,937	100 %
CopyAnArray.Main.main(String[])		80,937	100 %
Main.java:23 CopyAnArray.CopyByClone.start()		26,250	32 %
Main.java:30 CopyAnArray.CopyWithCopyOf.start()		21,781	27 %
Main.java:43 CopyAnArray.CopyWithForEach.start()		11,125	14 %
Main.java:37 CopyAnArray.CopyWithArrayCopy.start()		11,046	14 %
Main.java:17 CopyAnArray.CopyByForLoop.start()		10,640	13 %

The 1,4,5 methods kept an average time 10.5 seconds, the 4 method use slightly more the CPU by 1%. The others kept the gap as it is.

Windows 10, Local Machine – JAVA 11



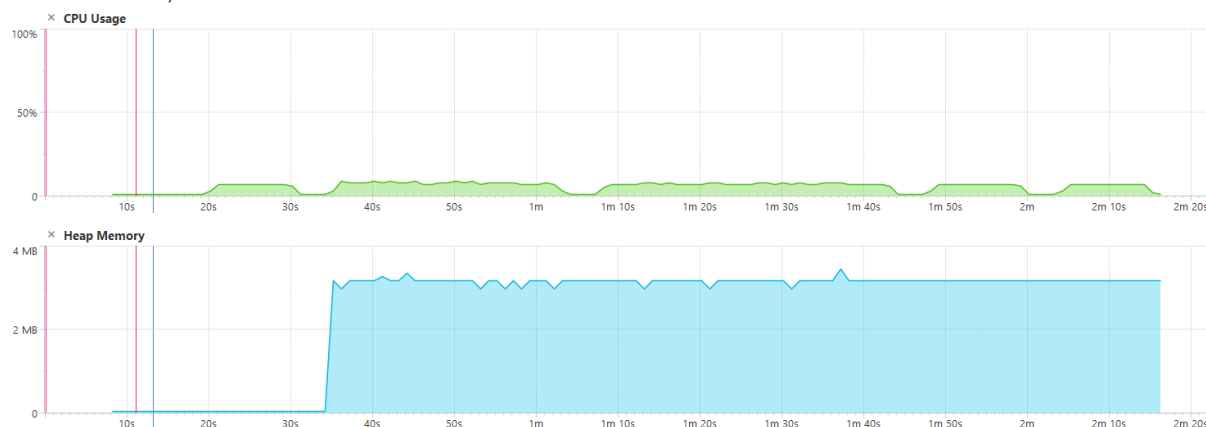
Same as Java8 when reaching 2,3 the Heap-memory is used, but and there is a **BIG** but, we see a drastic change in the memory usage, the max usage in Java8 is around 1.2GB and in

Java11 not even reaching 5MB, the explanation for this is probably since in Java 9 & 10 there are updates to the GC and more updates that related to memory allocation.

Call Tree		Time (ms)	
main	Group: 'main'	89,031	100 %
CopyAnArray.Main.main(String[])		89,031	100 %
Main.java:30	CopyAnArray.CopyWithCopyOf.start()	29,156	33 %
Main.java:23	CopyAnArray.CopyByClone.start()	28,515	32 %
Main.java:37	CopyAnArray.CopyWithArrayCopy.start()	11,328	13 %
Main.java:43	CopyAnArray.CopyWithForEach.start()	10,093	11 %
Main.java:17	CopyAnArray.CopyByForLoop.start()	9,859	11 %

The order did not change much, methods 4,5 change places an average of 10.5 seconds but the CPU usage was less for the 5 method in most cases or equal to 4 method.

Windows 10, Local Machine – JAVA 17

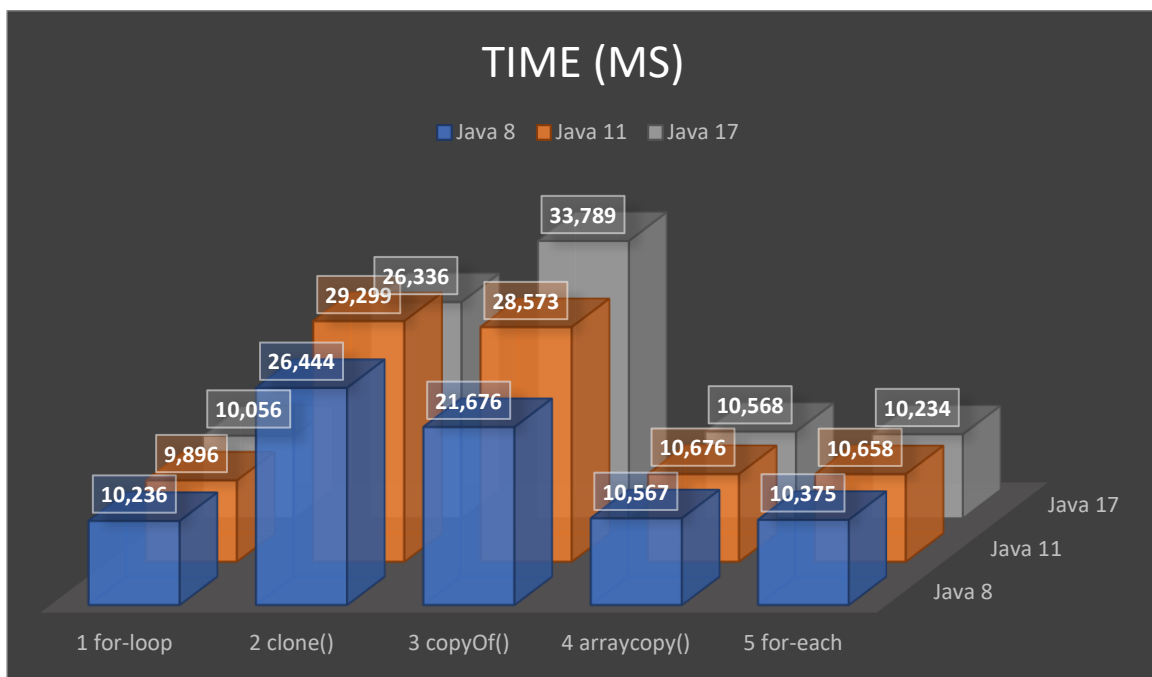
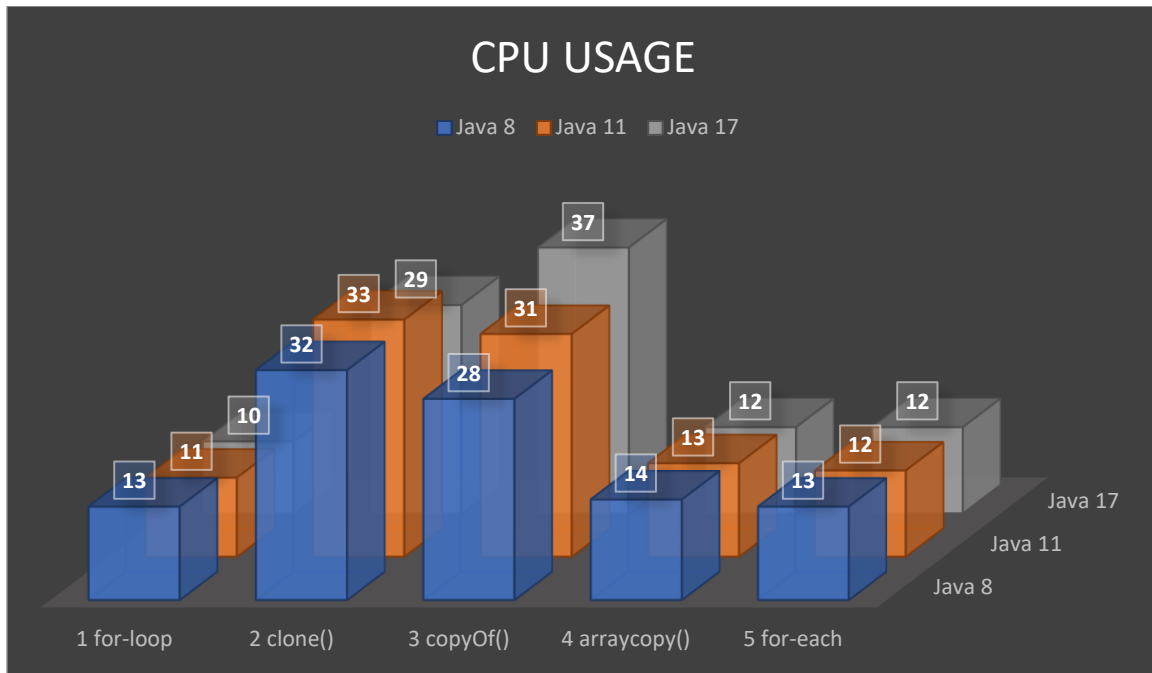


Another improvements in memory !

Call Tree		Time (ms)	
main	Group: 'main'	91,593	100 %
CopyAnArray.Main.main(String[])		91,593	100 %
Main.java:30	CopyAnArray.CopyWithCopyOf.start()	33,984	37 %
Main.java:23	CopyAnArray.CopyByClone.start()	26,234	29 %
Main.java:37	CopyAnArray.CopyWithArrayCopy.start()	11,046	12 %
Main.java:43	CopyAnArray.CopyWithForEach.start()	10,250	11 %
Main.java:17	CopyAnArray.CopyByForLoop.start()	10,031	11 %

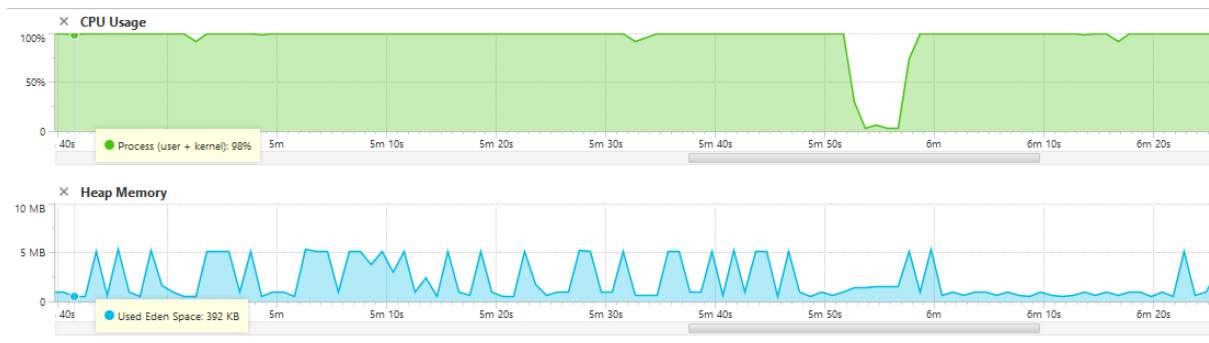
We see another change in events when running on Java17 this time the 3 method consumed much more than 2 method, in Java 8 & 11 the 2 method was in the top.

Conclusions:



Ubuntu18.04 VS Ubuntu22.04

Java 8



For both OS version the CPU usage and Heap-Memory were similar – it cannot make a real difference where we have only 1-core and 1-GB RAM.

We will check if there are improvements when running Java 11 and 17, and we will compare the

Java 11

Ubuntu18.04:

Call Tree			Time (ms)
main	Group: 'main'		180,805 100 %
CopyAnArray.Main.main(String[])			180,805 100 %
Main.java:23	CopyAnArray.CopyByClone.start()		55,893 31 %
Main.java:30	CopyAnArray.CopyWithCopyOf.start()		40,730 23 %
Main.java:43	CopyAnArray.CopyWithForEach.start()		29,627 16 %
Main.java:37	CopyAnArray.CopyWithArrayCopy.start()		27,971 15 %
Main.java:17	CopyAnArray.CopyByForLoop.start()		26,526 15 %

Ubuntu22.04:

Call Tree			Time (ms)
main	Group: 'main'		199,298 100 %
CopyAnArray.Main.main(String[])			199,298 100 %
Main.java:23	CopyAnArray.CopyByClone.start()		62,796 32 %
Main.java:30	CopyAnArray.CopyWithCopyOf.start()		51,969 26 %
Main.java:37	CopyAnArray.CopyWithArrayCopy.start()		28,350 14 %
Main.java:17	CopyAnArray.CopyByForLoop.start()		28,070 14 %
Main.java:43	CopyAnArray.CopyWithForEach.start()		28,058 14 %

We can see a big difference in running time – we suspect that in both version the kernel handles the program very well with the scheduler since it only had 1-core to use, but since 22.04 is more advance there probably more services running in the background since our profiler only inspect the JVM it did not look at other processes running with our Java program.

Java 17

Ubuntu18.04:

Call Tree				Time (ms)
main	Group: 'main'			180,275 100 %
CopyAnArray.Main.main(String[])				180,275 100 %
Main.java:29	CopyAnArray.CopyByClone.start()			58,317 32 %
Main.java:23	CopyAnArray.CopyWithCopyOf.start()			41,099 23 %
Main.java:37	CopyAnArray.CopyWithArrayCopy.start()			27,694 15 %
Main.java:43	CopyAnArray.CopyWithForEach.start()			26,833 15 %
Main.java:17	CopyAnArray.CopyByForLoop.start()			26,278 15 %

Ubuntu22.04:

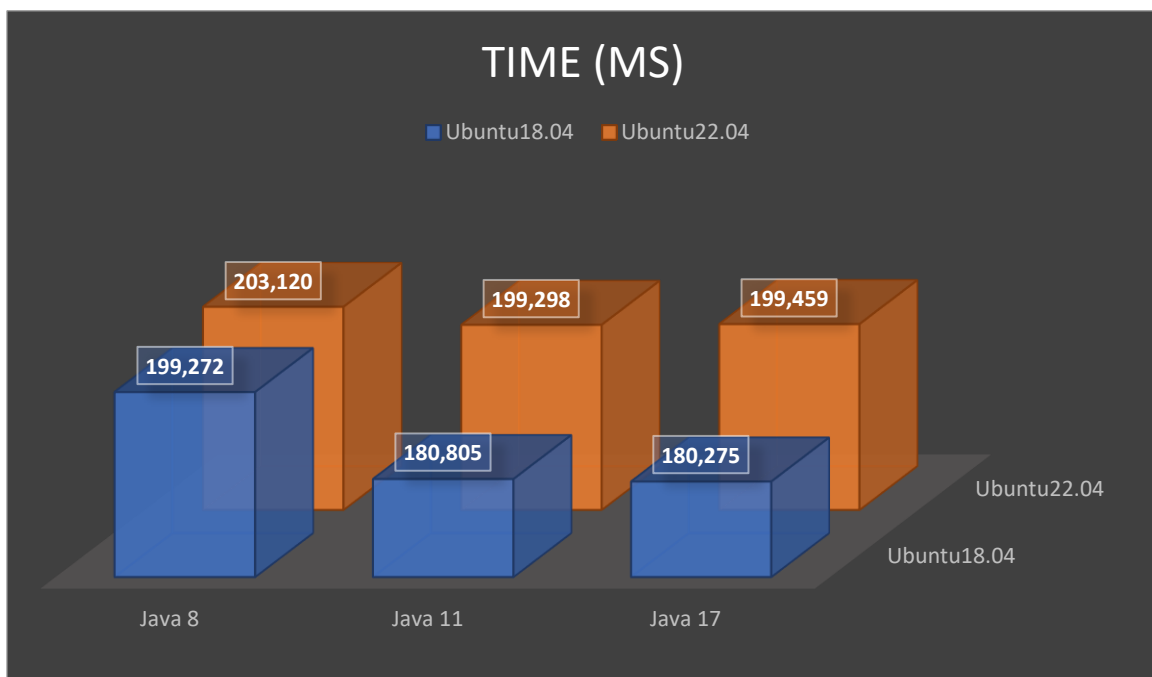
Call Tree				Time (ms)
main	Group: 'main'			192,459 100 %
CopyAnArray.Main.main(String[])				192,459 100 %
Main.java:29	CopyAnArray.CopyByClone.start()			60,402 31 %
Main.java:23	CopyAnArray.CopyWithCopyOf.start()			42,385 22 %
Main.java:43	CopyAnArray.CopyWithForEach.start()			33,537 17 %
Main.java:37	CopyAnArray.CopyWithArrayCopy.start()			28,230 15 %
Main.java:17	CopyAnArray.CopyByForLoop.start()			27,854 14 %

Same results here – regarding time differences.

In Java 8 there was not a significant difference in time, still 18.04 version was faster but since the JDK 8 dose not have the new GC in this version the scheduler did not work hard with the GC.

We can also see that the difference in the CPU usage for the methods was keeping the same order as windows 10 on our Local machine.

Conclusions:



We will run the program once more on our Local Machine with the different versions of the OS's but we will run it only using Java 17 JDK.

Local Machine – Ubuntu:

The performance charts look very similar to our Windows 10 results on version of the Ubuntu OS's, even the running time was with no significant difference.

There was a tiny improvement in running time, just to be exact but since our Ubuntu machine were only running our Java program, we suspect this is the cause of the “improvement”.

We did not find in this case a difference to point on to, on those 2 different OS version of Ubuntu the use of the Ubuntu18.04 was slower in general and less supportive but when running the Java program both version held their own, ran it and gave similar results.

Since Ubuntu22.04 was more convenient to use and faster in general when dealing with files and the IDE also the profiler itself, in our opinion it best to upgrade to a newer version as for now.

Java Streams – Powerful but which way ?

Well, whoever knows Java language should know Java Streams, you can really do a lot with it and with a few lines also, but we really could not ignore the fact that you can do the same thing in a different way – and of course get the same result.

Since our main subject is the Java language itself and not just the Stream API, we encourage you to find more ways of using Streams and get the same result – you check by yourself which is best.

We focus on 3 different ways to sum a list of object (Employee), one of the methods we divide it into 2 different ways, hence we got 4 terminal operations that conclude the same result.

1. Using Collectors

```
long integer = employees.stream() Stream<Employee>
    .map(employee -> (int) employee.age) Stream<Integer>
    .collect(Collectors.summingInt(Integer::intValue));
```

2. Reduce using lambda

```
long integer = employees.stream() Stream<Employee>
    .map(employee -> (int) employee.age) Stream<Integer>
    .reduce( identity: 0, (a, b) -> a + b);
```

3. Reduce using method reference

```
long integer = employees.stream() Stream<Employee>
    .map(employee -> (int) employee.age) Stream<Integer>
    .reduce( identity: 0, Integer::sum);
```

4. Mapping into an IntStream then summing it.

```
long integer = employees.stream() Stream<Employee>
    .mapToInt(employee -> (int) employee.age) IntStream
    .sum();
```

While trying to run our Streams testing, we encounter a problem since Java need to create the Stream object in order to use, and our Remote machines only have 1GB RAM we did not succeed with running the program.

```
ubuntu@ip-172-31-33-8:~/Streams$ java -jar Java8.jar
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
    at StreamTesting.Main.main(Main.java:15)
```

Hence, we will check the performance in our Local Machine.

Local Machine – Ubuntu:

When profiling this test case, we run into an issue – since the Stream API uses a lot of objects on parallel – when that's happens the profiler monitored and took samples A LOT, we had a really bad running time with it – so when you will look at the profiler pictures you can ignore the running time.

This is the real running time for each:

Java version: 1.8 press Enter to START when profiler is READY StreamReduceReference -> 7.126 StreamReduce -> 4.236 StreamSum -> 2.101 StreamCollect -> 4.253 StreamReduceReference -> 5.203 StreamReduce -> 5.136 StreamSum -> 2.177 StreamCollect -> 4.914 Process finished with exit code 0	Java version: 11.0 press Enter to START when profiler is READY StreamReduceReference -> 3.756 StreamReduce -> 4.03 StreamSum -> 1.911 StreamCollect -> 3.758 StreamReduceReference -> 4.516 StreamReduce -> 4.521 StreamSum -> 1.853 StreamCollect -> 4.083 Process finished with exit code 0	Java version: 17.0 press Enter to START when profiler is READY StreamReduceReference -> 3.473 StreamReduce -> 3.607 StreamSum -> 1.838 StreamCollect -> 3.259 StreamReduceReference -> 4.017 StreamReduce -> 4.115 StreamSum -> 1.773 StreamCollect -> 3.64 Process finished with exit code 0
---	---	---

As we can see there is a big improvement since Java 11 with the Stream API, and a slightly improvement when running Java 17 – this is probably another upgrade to the API that we did not know of.

There was no improvement when running on different versions of Ubuntu – the above results are from the 18.04 version. We will stick with this version to investigate Streams.

Java 8

Call Tree		Time (ms)	
main	Group: 'main'	725,392	100 %
StreamTesting.Main.main(String[])		725,392	100 %
Main.java:39 StreamTesting.StreamReduce.start(List)		290,312	40 %
StreamReduce.java:12 <...> java.util.stream.ReferencePipeline.reduce(Object, BinaryOperator)		290,312	40 %
<7 calls> ReduceOps.java:80 StreamTesting.StreamReduce\$\$Lambda\$20.apply(Object, Object)		150,400	21 %
<6 calls> ReferencePipeline.java:193 StreamTesting.StreamReduce\$\$Lambda\$19.apply(Object)		139,672	19 %
Main.java:31 StreamTesting.StreamReduceReference.start(List)		276,580	38 %
StreamReduceReference.java:13 <...> java.util.stream.ReferencePipeline.reduce(Object, BinaryOperator)		276,540	38 %
<6 calls> ReferencePipeline.java:193 StreamTesting.StreamReduceReference\$\$Lambda\$15.apply(Object)		144,768	20 %
<7 calls> ReduceOps.java:80 StreamTesting.StreamReduceReference\$\$Lambda\$16.apply(Object, Object)		131,772	18 %
StreamReduceReference.java:13 <...> java.lang.invoke.MethodHandleNatives.linkCallSite(Object, Object, Object, Object, Object, Object, Object[])		40	0 %
Main.java:56 StreamTesting.StreamCollect.start(List)		154,096	21 %
StreamCollect.java:14 <...> java.util.stream.ReferencePipeline.collect(Collector)		154,080	21 %
<6 calls> ReferencePipeline.java:193 StreamTesting.StreamCollect\$\$Lambda\$26.apply(Object)		154,004	21 %
StreamTesting.StreamCollect.lambda\$start\$0(Employee)		154,004	21 %
StreamCollect.java:13 java.lang.Double.valueOf(double)		154,004	21 %
Double.java:519		154,004	21 %
StreamCollect.java:14 <...> java.util.stream.Collectors.summingDouble(ToDoubleFunction)		16	0 %
Main.java:48 StreamTesting.StreamSum.start(List)		4,240	1 %
StreamSum.java:11 <...> java.util.stream.DoublePipeline.sum()		4,160	1 %
StreamSum.java:9 <...> java.util.Collection.stream()		80	0 %

We will ignore the time factor and only look on the CPU usage of each class.start(), as we can see these results took a lot more time than without the profiler but since the 2 top method that use the `reduce` terminal are above we can be sure that it is less recommended to use and when summing Numbers, ***you should use Double\Int Streams.***

Java 11

Call Tree		Time (ms)	
main	Group: 'main'	1,308,904	100 %
StreamTesting.Main.main(String[])		1,308,904	100 %
Main.java:39	StreamTesting.StreamReduce.start(List)	524,352	40 %
StreamReduce.java:12	java.util.stream.ReferencePipeline.reduce(Object, BinaryOperator)	524,352	40 %
<7 calls>	ReduceOps.java:80	272,440	21 %
<6 calls>	ReferencePipeline.java:195	251,912	19 %
Main.java:31	StreamTesting.StreamReduceReference.start(List)	500,764	38 %
StreamReduceReference.java:13	java.util.stream.ReferencePipeline.reduce(Object, BinaryOperator)	500,764	38 %
<6 calls>	ReferencePipeline.java:195	253,452	19 %
<7 calls>	ReduceOps.java:80	247,184	19 %
Main.java:56	StreamTesting.StreamCollect.start(List)	277,336	21 %
StreamCollect.java:14	java.util.stream.ReferencePipeline.collect(Collector)	277,336	21 %
<6 calls>	ReferencePipeline.java:195	277,288	21 %
StreamTesting.StreamCollect.lambda\$start\$0(Employee)		277,288	21 %
StreamCollect.java:13	java.lang.Double.valueOf(double)	277,288	21 %
Double.java:524		277,288	21 %
Main.java:48	StreamTesting.StreamSum.start(List)	5,892	0 %
StreamSum.java:11	java.util.stream.DoublePipeline.sum()	5,888	0 %
StreamSum.java:9	java.lang.invoke.MethodHandleNatives.linkMethodHandleConstant(Class, int, Class, String, O	4	0 %

This time when monitoring with the profiler it took almost double the time, in order to really understand why the time differences is this vast we need to investigate the Stream API much more and understand the improvements that has been done between those versions, we can see and say briefly that were changes in the method that we used as you can see in the pictures.

Java 17

Call Tree		Time (ms)	
main	Group: 'main'	1,562,540	100 %
StreamTesting.Main.main(String[])		1,562,540	100 %
Main.java:39	StreamTesting.StreamReduce.start(List)	623,180	40 %
StreamReduce.java:12	java.util.stream.ReferencePipeline.reduce(Object, BinaryOperator)	623,180	40 %
<7 calls>	ReduceOps.java:80	323,784	21 %
<6 calls>	ReferencePipeline.java:197	299,192	19 %
Main.java:31	StreamTesting.StreamReduceReference.start(List)	611,128	39 %
StreamReduceReference.java:13	java.util.stream.ReferencePipeline.reduce(Object, BinaryOperator)	611,128	39 %
<7 calls>	ReduceOps.java:80	316,480	20 %
<6 calls>	ReferencePipeline.java:197	294,648	19 %
Main.java:56	StreamTesting.StreamCollect.start(List)	322,240	21 %
StreamCollect.java:14	java.util.stream.ReferencePipeline.collect(Collector)	322,240	21 %
<6 calls>	ReferencePipeline.java:197	322,220	21 %
StreamTesting.StreamCollect.lambda\$start\$0(Employee)		322,220	21 %
StreamCollect.java:13	java.lang.Double.valueOf(double)	322,220	21 %
Double.java:632		322,220	21 %
Main.java:48	StreamTesting.StreamSum.start(List)	5,868	0 %
StreamSum.java:11	java.util.stream.DoublePipeline.sum()	5,868	0 %

It took even more time around 250 seconds, we suspect the same reason as before there are improvements that need to be dug into deeper and understand how the API really works – we can also see changes in the behavior of the functions since Java 11.

We can conclude that the ‘Reduce’ terminal take more of your Java program and if the minor milliseconds and the usage of the CPU is crucial for you, investigate more and use the most efficient way for your program.

The Stream API is a subject to write a paper by itself and since it is not our main subject we will continue with other cases.

Record vs. Class

Record is a special class that we got to know in Java 14, it used to replace a class that act only as a plain data carrier, when you need to keep records in your data base for example, you can use class that way.

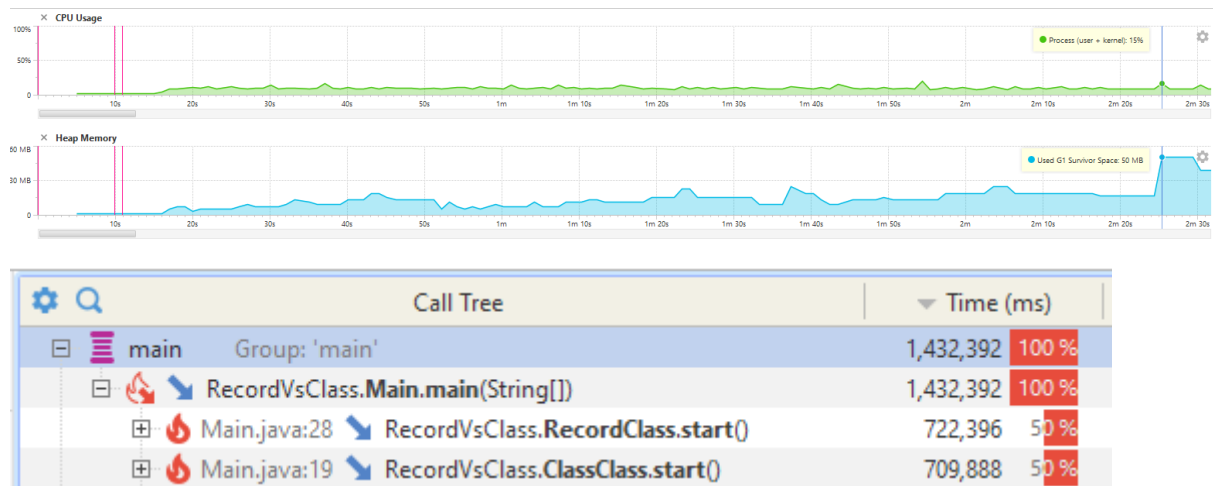
Since Java 14, Record is the new way of doing it, I suppose once you get used it, it is more convenient and your code will be much more readable and will avoid boilerplate code, but is it efficient ?

(Record was Standardized since Java 16).¹

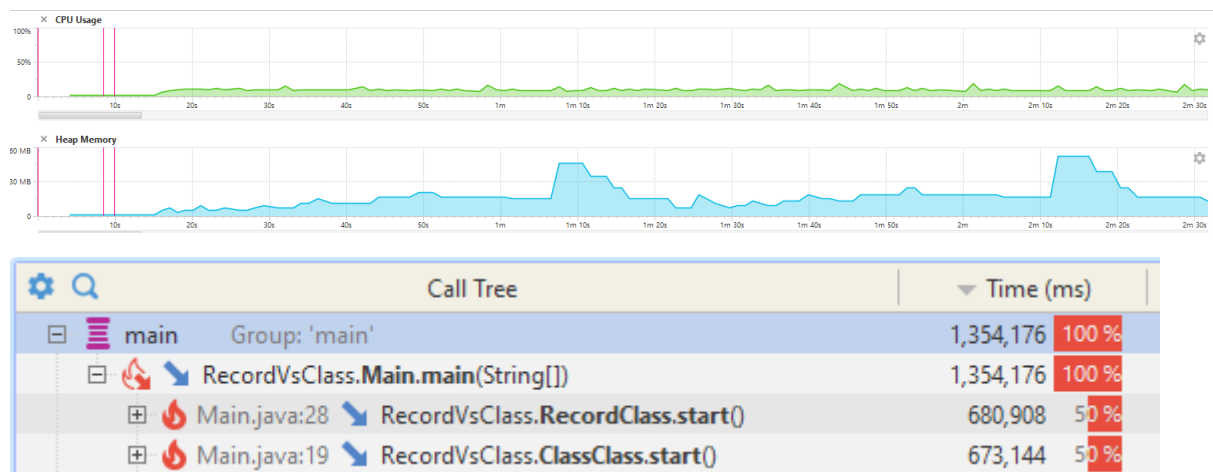
Since we only comparing LTS versions we only run it on Java 17 to see if the new Java feature “Record special class” is recommended or just an improvement for readability.

We run our testing program on Ubuntu to compare also between the OS versions.

Ubuntu18.04



Ubuntu22.04



¹ For more information – [Features in each version](#)

We can see there is a difference between the OS running time almost a 100 seconds when both Java program were monitored and sampled by the profiler, we can also see in the performance charts that on the 22.04 version the Heap Memory were used early in time a similar behavior for both OS`s, but since the latest version were faster, we can see it earlier in the chart.

Let`s get back to our Java comparison, we see what we except the Record class took a tiny bit more time, and equal amount of the CPU of our Main-Thread, since the Record class has in its background a final class that it is pointing at, we think this is the factor to time differences – but those are the real running time of the program without using a profiler.

```
Java version: 17.0
press Enter to START when profiler is READY

ClassClass -> 4.56
RecordClass -> 4.199
ClassClass -> 4.249
RecordClass -> 4.176
ClassClass -> 4.233
RecordClass -> 4.37
ClassClass -> 4.253
RecordClass -> 4.215
ClassClass -> 4.357
RecordClass -> 4.36

Process finished with exit code 0
```

Ubuntu 18.04

```
Java version: 17.0
press Enter to START when profiler is READY

ClassClass -> 4.301
RecordClass -> 4.087
ClassClass -> 4.102
RecordClass -> 4.02
ClassClass -> 4.105
RecordClass -> 3.878
ClassClass -> 3.92
RecordClass -> 3.94
ClassClass -> 3.898
RecordClass -> 4.023
```

Ubuntu 22.04

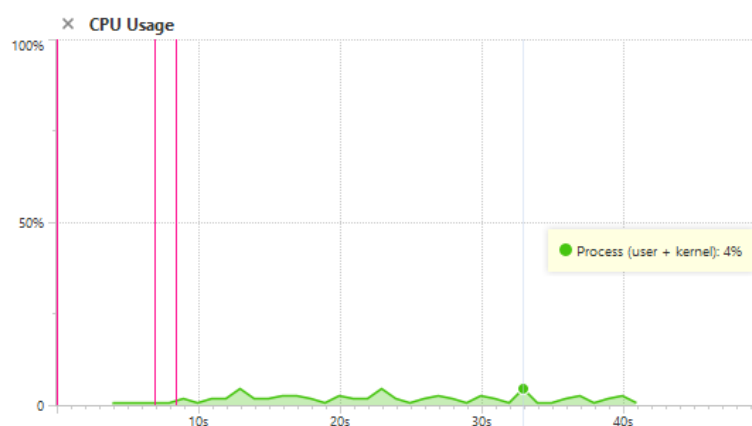
What should I do Switch-Case or If and If else ?

Every programmer when dealing with multiple inputs that each one need to be handled differently, had that thought in mind – is too much if-else, should I do switch-case ?

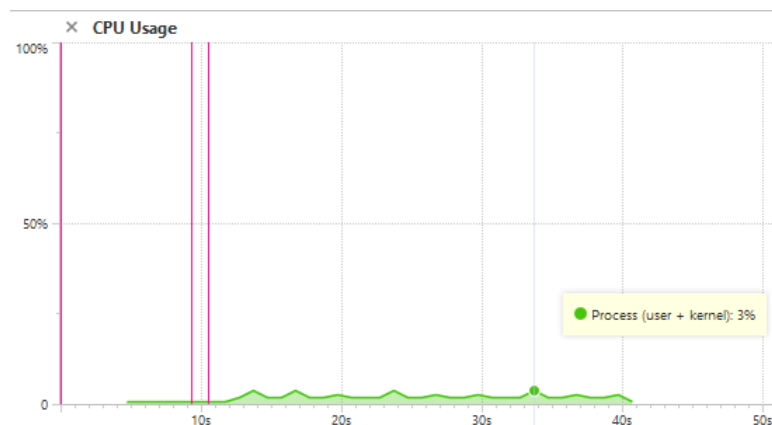
Well switch case is usually much more readable especially when you're dealing with cases that need to handle the input the same way, with if-else it will be more complicated to achieve it, but in general what is most efficient way ?

Same as the Record case, since we had an improvement in the switch-case functionality that Java 12-13 introduced to us, we only run this case on Java 17 on Ubuntu.

Ubuntu 18.04



Ubuntu 22.04



The usage seems better on the latest version of Ubuntu, not by far, we can guess that if we will really take this test to the extreme, a significant difference will result.

There is a slightly better running time for the Ubuntu 22.04 (LATEST) of 200 ms, but this is not a great deal when talking about performance let look at our calling tree of the Main-Thread.

Ubuntu 18.04:

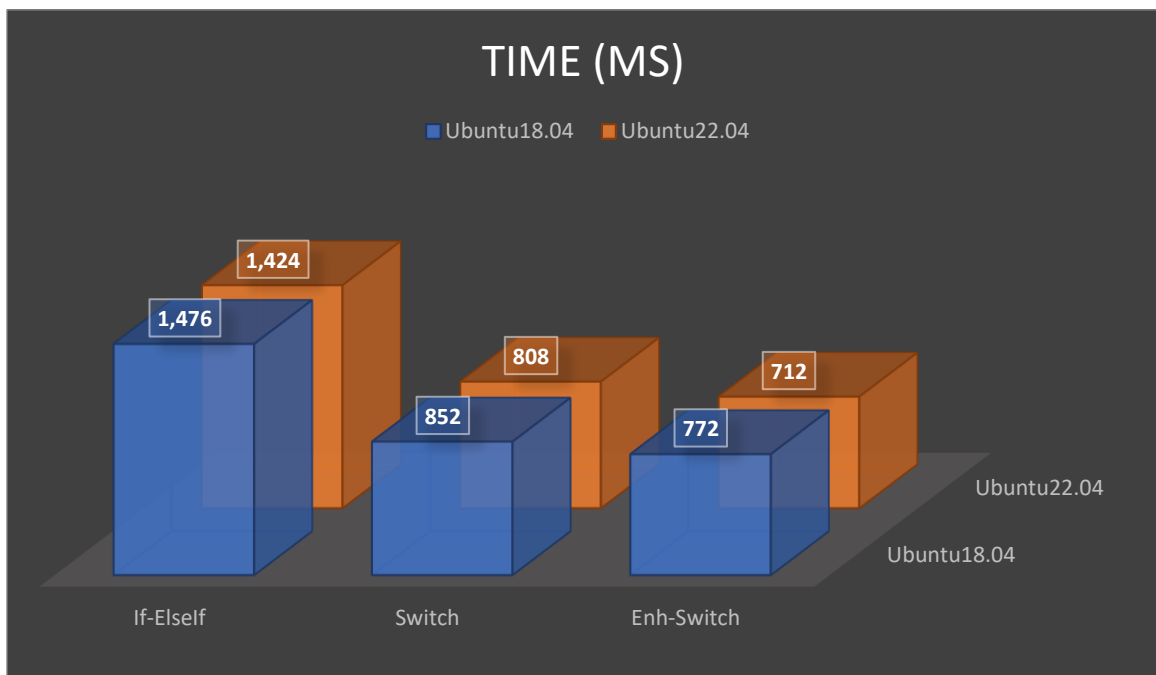
Call Tree				Time (ms)	
main	Group: 'main'			3,224	100 %
SwitchVslfElse.Main.main(String[])				3,224	100 %
Main.java:25	SwitchVslfElse.IfElseCases.start(String[])			1,476	45 %
Main.java:31	SwitchVslfElse.SwitchCases.start(String[])			852	26 %
Main.java:38	SwitchVslfElse.SwitchEnhanced.start(String[])			772	24 %

Ubuntu 22.04:

Call Tree				Time (ms)	
main	Group: 'main'			3,020	100 %
SwitchVslfElse.Main.main(String[])				3,020	100 %
Main.java:25	SwitchVslfElse.IfElseCases.start(String[])			1,424	47 %
Main.java:31	SwitchVslfElse.SwitchCases.start(String[])			808	27 %
Main.java:38	SwitchVslfElse.SwitchEnhanced.start(String[])			712	24 %

In both OS versions we can clearly see that when dealing with a related conditions that a Switch-case will fit – better to use it.

Also, if you are running on Java 13 and higher it is better to use the enhanced switch.



Using CONSTANTS – Enum or Static final ?

When developing a big project, and you want to have a clean and readable code you need to have a lot of *CONSTANTS*.

The old style is to use Strings as a static members in one class of course those members need to have some relation considering they are in the same class, but sometimes you have only one String that represent something and that is it, you probably will not open a dedicated class just for one String, since Java 5 Enumerations are available to us in Java and dedicated class can be created to all those Enum`s.

But the special question needs to be asked, which is best ?

Same as the Stream performance testing, we run out of memory again.
the performance testing will be on our remote machine.

```
ubuntu@ip-172-31-33-8:~/EnumStaticFinal$ java -jar Java8.jar
Java version: 1.8
press Enter to START when profiler is READY
[YourKit Java Profiler 2022.3-b107] Log file: /home/ubuntu/.yjp/log/Java8-23469.log

java.lang.OutOfMemoryError: Java heap space
Dumping heap to java_pid23469.hprof ...
Heap dump file created [212133120 bytes in 1.178 secs]
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
    at java.util.Arrays.copyOf(Arrays.java:3210)
    at java.util.Arrays.copyOf(Arrays.java:3181)
    at java.util.ArrayList.grow(ArrayList.java:267)
    at java.util.ArrayList.ensureExplicitCapacity(ArrayList.java:241)
    at java.util.ArrayList.ensureCapacityInternal(ArrayList.java:233)
    at java.util.ArrayList.add(ArrayList.java:464)
    at EnumVsStaticFinal.UseStaticFinal.start(UseStaticFinal.java:49)
    at EnumVsStaticFinal.Main.main(Main.java:13)
```

Ubuntu 18.04

Since the Enum-class has pointer to another class we thought for sure that in performance the Static-Final-class will do better, but this is not really the case.

Java 8

Call Tree			Time (ms)	
main	Group: 'main'		32,616	100 %
EnumVsStaticFinal.Main.main(String[])			32,616	100 %
Main.java:21	EnumVsStaticFinal.UseStaticFinal.start()		16,640	51 %
Main.java:29	EnumVsStaticFinal.UseEnum.start()		15,844	49 %

We can see tiny bit of time and CPU usage difference and that is a surprise.

Maybe and the newer versions of Java we will see a different result since the Enum class point another class who uses Static-Finals.

Java 11

Call Tree			Time (ms)	
main	Group: 'main'		90,896	100 %
EnumVsStaticFinal.Main.main(String[])			90,896	100 %
Main.java:21	EnumVsStaticFinal.UseStaticFinal.start()		45,432	50 %
Main.java:29	EnumVsStaticFinal.UseEnum.start()		45,336	50 %

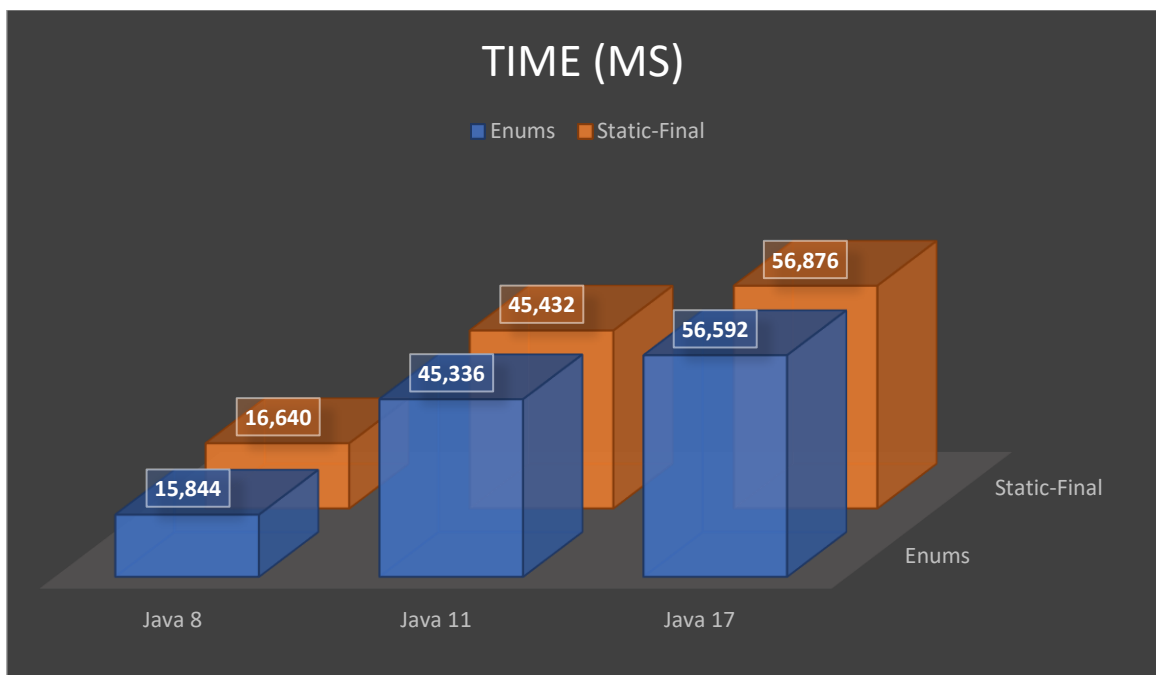
The time factor is irrelevant in this version and the Java 17 since the profiler took a heavy roll on these versions since the GC is parallel and it is being monitored as well.

But we can see the Enum-class winning again

Java 17

Call Tree			Time (ms)	
main	Group: 'main'		113,604	100 %
EnumVsStaticFinal.Main.main(String[])			113,604	100 %
Main.java:25	EnumVsStaticFinal.UseStaticFinal.start()		56,876	50 %
Main.java:33	EnumVsStaticFinal.UseEnum.start()		56,592	50 %

Enum-class is better in all versions if you ever thought of using it – you should.



Ubuntu 22.04

Now we will check if there are improvements when using the latest version of Ubuntu

Java 8

Call Tree			Time (ms)	
main	Group: 'main'		31,672	100 %
EnumVsStaticFinal.Main.main(String[])			31,672	100 %
Main.java:21	EnumVsStaticFinal.UseStaticFinal.start()		16,052	51 %
Main.java:29	EnumVsStaticFinal.UseEnum.start()		15,480	49 %

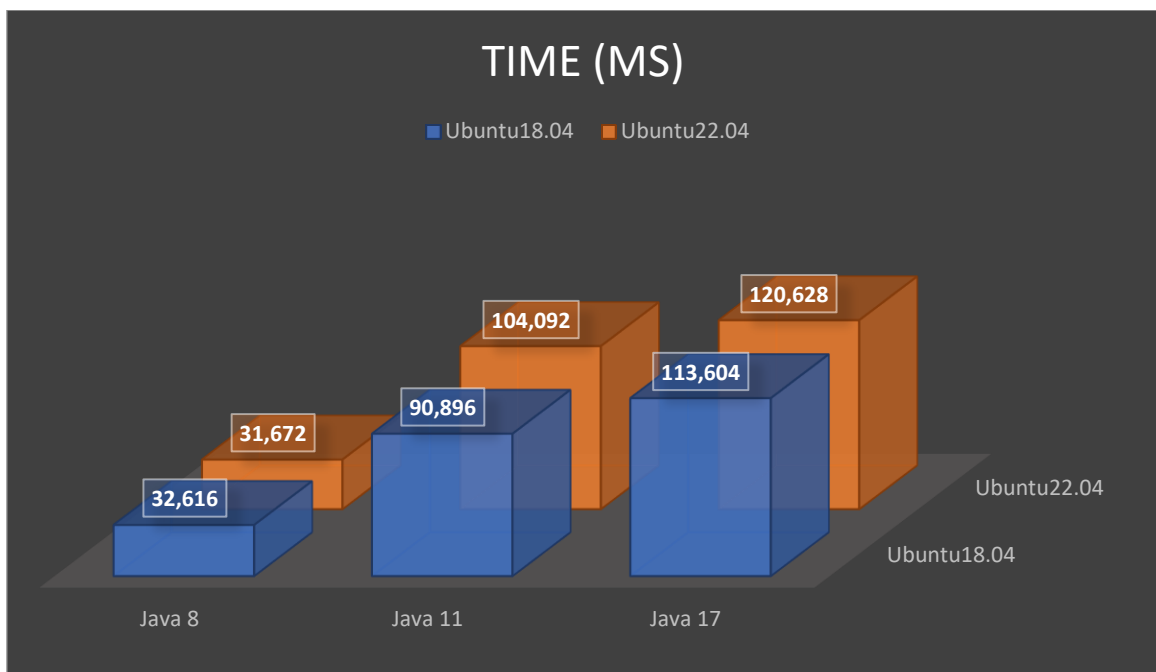
Java 11

Call Tree			Time (ms)	
main	Group: 'main'		104,092	100 %
EnumVsStaticFinal.Main.main(String[])			104,092	100 %
Main.java:25	EnumVsStaticFinal.UseStaticFinal.start()		52,316	50 %
Main.java:33	EnumVsStaticFinal.UseEnum.start()		51,624	50 %

Java 17

Call Tree			Time (ms)	
main	Group: 'main'		120,628	100 %
EnumVsStaticFinal.Main.main(String[])			120,628	100 %
Main.java:33	EnumVsStaticFinal.UseEnum.start()		61,968	51 %
Main.java:25	EnumVsStaticFinal.UseStaticFinal.start()		58,540	49 %

Conclusions



There are no significant differences

Should I use a Stream or a For loop or ForEach ?

Every programmer had to iterate over a data structure at least once in day-day developing did you ever ask yourself how to iterate over the data structure is preferable ? we ask ourselves for you.

Well having an array, you can either loop over it with 2 different ways in Java or you can use the Stream API that Java has to offer since Java 8, sometimes you just iterate over it with a simple task like to sum all its elements, and sometimes it is a bit more complex, and you need to filter some of the element that does not qualify your terms.

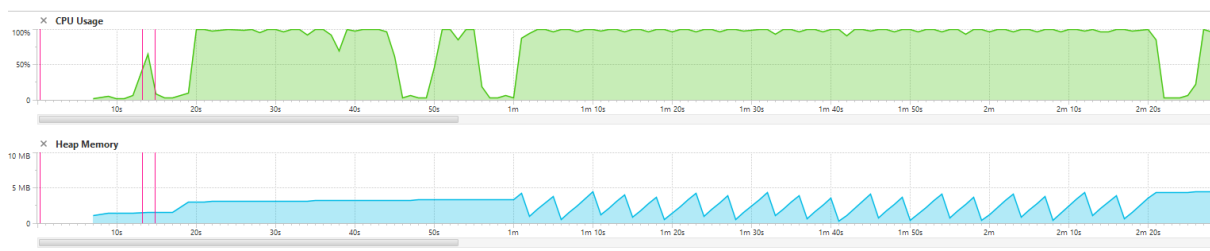
We only run these test scenarios on our remote machine, to take this performance to the extreme without any parallel or something like that, using only 1-core and 1-Main Thread to run it all.

Starting by just a simple method of summing an array without any conditions or filtering, and since when usually iterating over an array, we want to filter some of the elements and summing or doing something else on the qualified elements, so we also test this case, so each scenario is divided into 2 cases -

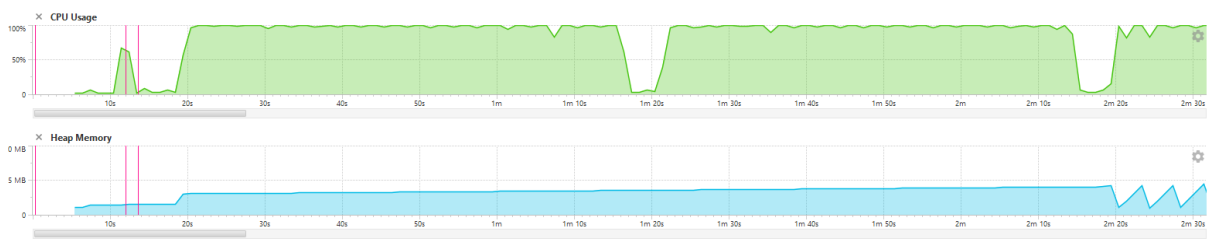
1. Summing all the elements of the array by iterating over it.
2. Summing all the elements that qualify 2 simple conditions.

Ubuntu18.04 – Remote Machine – EC2:

Java 8



with conditions:



we can see a significant difference between those two runs, in both images the first green block is a ForEach loop after that a For-Loop, then using the Stream API (when the Heap-Memory starting to look like zigzag).

the CPU usage images will make things clearer.

Call Tree			Time (ms)	
main	Group: 'main'		309,853	100 %
IterateOver.Main.main(String[])			309,853	100 %
Main.java:33	IterateOver.UsingStream.start()		210,167	68 %
Main.java:20			47,817	15 %
Main.java:20	IterateOver.ForEachLoop.start()		33,178	11 %
Main.java:26			13,909	4 %
Main.java:26	IterateOver.ForLoop.start()		4,728	2 %
Main.java:36	<...> java.util.concurrent.TimeUnit.sleep(long)		28	0 %
Main.java:23	<...> java.util.concurrent.TimeUnit.sleep(long)		16	0 %
Main.java:29	<...> java.util.concurrent.TimeUnit.sleep(long)		8	0 %

With conditions:

Call Tree			Time (ms)	
main	Group: 'main'		625,711	100 %
IterateOverWithCond.Main.main(String[])			625,711	100 %
Main.java:33	IterateOverWithCond.UsingStream.start()		275,013	44 %
Main.java:26			126,875	20 %
Main.java:20			101,948	16 %
Main.java:20	IterateOverWithCond.ForEachLoop.start()		75,074	12 %
Main.java:26	IterateOverWithCond.ForLoop.start()		46,751	7 %
Main.java:22	<...> java.io.PrintStream.println(String)		20	0 %
Main.java:29	<...> java.util.concurrent.TimeUnit.sleep(long)		12	0 %
Main.java:36	<...> java.util.concurrent.TimeUnit.sleep(long)		12	0 %
Main.java:23	<...> java.util.concurrent.TimeUnit.sleep(long)		4	0 %

Since the results are not so clear let's explain what we are seeing here, we will add each main line that we are seeing to look at it as one operation that our Main-Thread executed, meaning – Main.java<line-number> - each line-number will be examine as one operation. So we will be looking on 3 main line-numbers – 20, 26, 33
on line number 20 – ForEach method were used
on line number 26 – For-Loop method were used
on line number 33 – Stream API were used.

The total difference outcome between the “with-conditions” and a regular iterations are very visible, and we can that Stream API did better results when he sums less elements after filtering with given conditions, the other methods did better without the conditions.

Java 11

The performance charts when looking on the raw details, very similar to the Java 8 performance charts above, but we see a significant change in the CPU usage when diving into the details of the Main-Thread.

Call Tree		Time (ms)
main	Group: 'main'	283,884 100 %
IterateOver.Main.main(String[])		283,884 100 %
Main.java:33	IterateOver.UsingStream.start()	216,389 76 %
Main.java:20		33,376 12 %
Main.java:20	IterateOver.ForEachLoop.start()	32,866 12 %
Main.java:26	IterateOver.ForLoop.start()	812 0 %
Main.java:26		412 0 %
Main.java:36	<...> java.util.concurrent.TimeUnit.sleep(long)	16 0 %
Main.java:23	<...> java.util.concurrent.TimeUnit.sleep(long)	12 0 %

With conditions:

Call Tree		Time (ms)
main	Group: 'main'	584,633 100 %
IterateOverWithCond.Main.main(String[])		584,633 100 %
Main.java:33	IterateOverWithCond.UsingStream.start()	297,105 51 %
Main.java:20		90,656 16 %
Main.java:26	IterateOverWithCond.ForLoop.start()	78,306 13 %
Main.java:26		66,584 11 %
Main.java:20	IterateOverWithCond.ForEachLoop.start()	51,929 9 %
Main.java:36	<...> java.util.concurrent.TimeUnit.sleep(long)	32 0 %
Main.java:23	<...> java.util.concurrent.TimeUnit.sleep(long)	16 0 %
Main.java:29	<...> java.util.concurrent.TimeUnit.sleep(long)	4 0 %

The first bold difference is the time in both programs in Java 11 it took less time to run both programs, the second significant difference is that when no conditions are in count For-Loop sums the element in around 1 second.

Java 17

No significant differences when looking on the raw details on the performance charts as before but let`s dig in our Main-Thread to find it.

Call Tree		Time (ms)
main	Group: 'main'	283,985 100 %
IterateOver.Main.main(String[])		283,985 100 %
Main.java:33	IterateOver.UsingStream.start()	217,831 77 %
Main.java:20		43,923 15 %
Main.java:20	IterateOver.ForEachLoop.start()	22,158 8 %
Main.java:36	<...> java.util.concurrent.TimeUnit.sleep(long)	43 0 %
Main.java:23	<...> java.util.concurrent.TimeUnit.sleep(long)	11 0 %
Main.java:26	IterateOver.ForLoop.start()	8 0 %
Main.java:29	<...> java.util.concurrent.TimeUnit.sleep(long)	8 0 %

With conditions:

Call Tree			Time (ms)	
main	Group: 'main'		458,846	100 %
IterateOverWithCond.Main.main(String[])			458,846	100 %
Main.java:33	IterateOverWithCond.UsingStream.start()		158,694	35 %
Main.java:20	IterateOverWithCond.ForEachLoop.start()		79,505	17 %
Main.java:26	IterateOverWithCond.ForLoop.start()		75,881	17 %
Main.java:26			74,271	16 %
Main.java:20			70,423	15 %
Main.java:35	<...> java.io.PrintStream.println(String)		19	0 %
Main.java:36	<...> java.util.concurrent.TimeUnit.sleep(long)		19	0 %
Main.java:23	<...> java.util.concurrent.TimeUnit.sleep(long)		16	0 %
Main.java:29	<...> java.util.concurrent.TimeUnit.sleep(long)		15	0 %

Java 17 continue to surprise us with almost no time at all using For-Loop just to simply sums up all the elements, and an overall improvements in time when summing elements using conditions.

Ubuntu22.04 – Remote Machine – EC2:

Java 8



On both OS's it is look very much alike, the green color is the 22.04 version also the cyan color on the Heap-Memory, let`s have a look on the CPU usage.

Call Tree		Time (ms)	
main	Group: 'main'	313,990	100 %
IterateOver.Main.main(String[])		313,990	100 %
Main.java:33	IterateOver.UsingStream.start()	211,165	67 %
Main.java:20		47,646	15 %
Main.java:20	IterateOver.ForEachLoop.start()	32,999	11 %
Main.java:26		19,443	6 %
Main.java:26	IterateOver.ForLoop.start()	2,680	1 %
Main.java:36	<...> java.util.concurrent.TimeUnit.sleep(long)	24	0 %
Main.java:22	<...> java.io.PrintStream.println(String)	20	0 %
Main.java:23	<...> java.util.concurrent.TimeUnit.sleep(long)	8	0 %
Main.java:29	<...> java.util.concurrent.TimeUnit.sleep(long)	4	0 %

With conditions:

Call Tree		Time (ms)	
main	Group: 'main'	620,476	100 %
IterateOverWithCond.Main.main(String[])		620,476	100 %
Main.java:33	IterateOverWithCond.UsingStream.start()	271,747	44 %
Main.java:26		129,096	21 %
Main.java:20		100,472	16 %
Main.java:20	IterateOverWithCond.ForEachLoop.start()	72,205	12 %
Main.java:26	IterateOverWithCond.ForLoop.start()	46,886	8 %
Main.java:36	<...> java.util.concurrent.TimeUnit.sleep(long)	48	0 %
Main.java:22	<...> java.io.PrintStream.println(String)	20	0 %

We can safely say it took both OS's around the same time to execute both programs on Java 8.

on 22.04 a tiny bit longer.

Java 11

Call Tree		Time (ms)	
main	Group: 'main'	284,212	100 %
IterateOver.Main.main(String[])		284,212	100 %
Main.java:33	IterateOver.UsingStream.start()	216,805	76 %
Main.java:20		44,363	16 %
Main.java:20	IterateOver.ForEachLoop.start()	22,171	8 %
Main.java:26		824	0 %
Main.java:36	<...> java.util.concurrent.TimeUnit.sleep(long)	24	0 %
Main.java:26	IterateOver.ForLoop.start()	12	0 %
Main.java:29	<...> java.util.concurrent.TimeUnit.sleep(long)	7	0 %
Main.java:23	<...> java.util.concurrent.TimeUnit.sleep(long)	4	0 %

With conditions:

Call Tree			Time (ms)
main	Group: 'main'		590,869 100 %
IterateOverWithCond.Main.main(String[])			590,869 100 %
Main.java:33	IterateOverWithCond.UsingStream.start()		297,310 50 %
Main.java:26	IterateOverWithCond.ForLoop.start()		78,765 13 %
Main.java:20	IterateOverWithCond.ForEachLoop.start()		77,246 13 %
Main.java:26			69,287 12 %
Main.java:20			68,196 12 %
Main.java:36	<...> java.util.concurrent.TimeUnit.sleep(long)		24 0 %
Main.java:23	<...> java.util.concurrent.TimeUnit.sleep(long)		20 0 %
Main.java:29	<...> java.util.concurrent.TimeUnit.sleep(long)		20 0 %

Similar results like the 18.04 version but took a bit longer as before.

Java 17

No significant differences when looking on the raw details on the performance charts as before but let's dig in our Main-Thread to find it.

Call Tree			Time (ms)
main	Group: 'main'		282,723 100 %
IterateOver.Main.main(String[])			282,723 100 %
Main.java:33	IterateOver.UsingStream.start()		216,809 77 %
Main.java:20			33,243 12 %
Main.java:20	IterateOver.ForEachLoop.start()		32,609 12 %
Main.java:36	<...> java.util.concurrent.TimeUnit.sleep(long)		31 0 %
Main.java:23	<...> java.util.concurrent.TimeUnit.sleep(long)		15 0 %
Main.java:26	IterateOver.ForLoop.start()		12 0 %

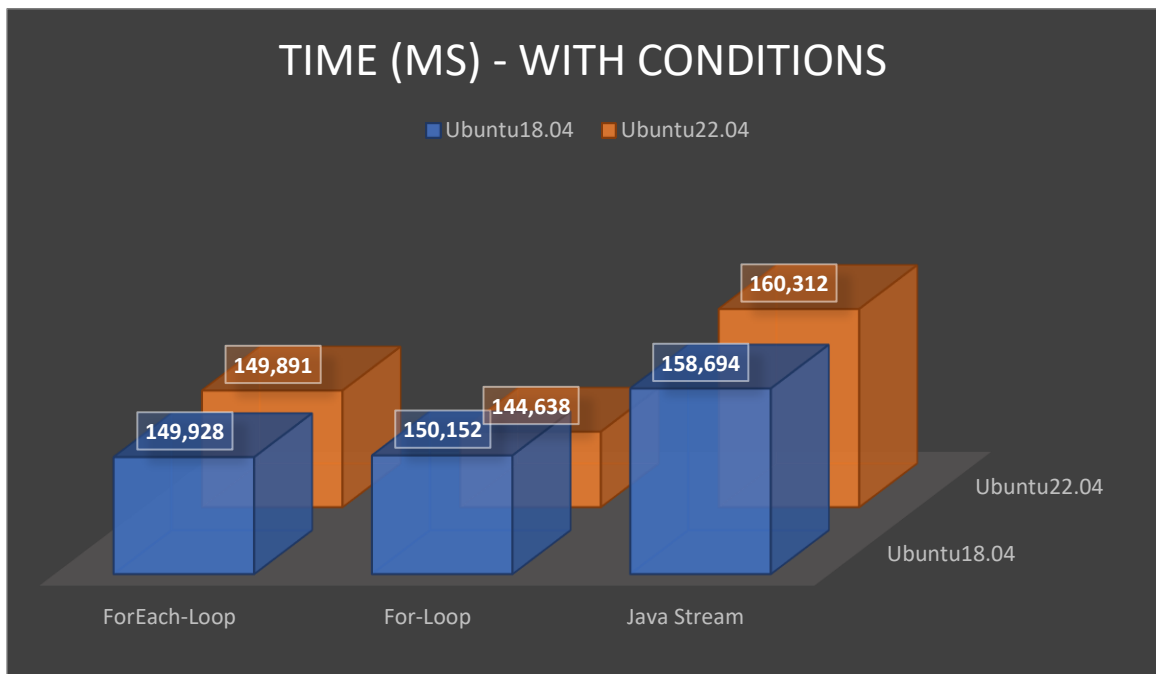
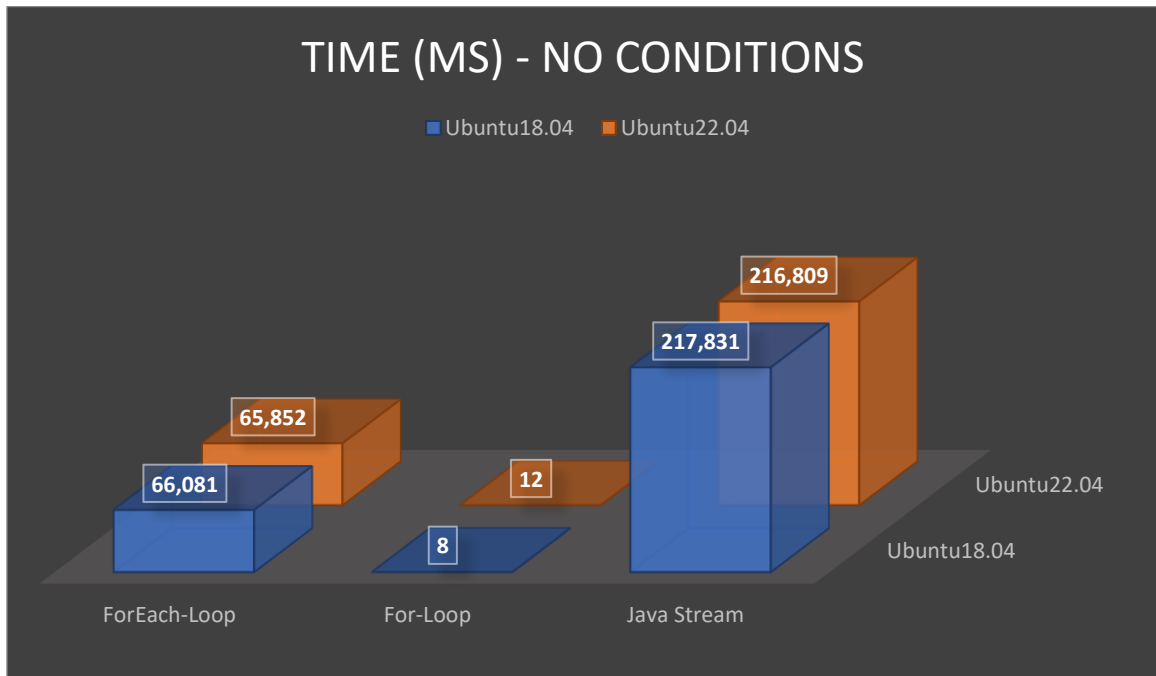
With conditions:

Call Tree			Time (ms)
main	Group: 'main'		454,968 100 %
IterateOverWithCond.Main.main(String[])			454,968 100 %
Main.java:33	IterateOverWithCond.UsingStream.start()		160,312 35 %
Main.java:20	IterateOverWithCond.ForEachLoop.start()		79,736 18 %
Main.java:26	IterateOverWithCond.ForLoop.start()		74,483 16 %
Main.java:20			70,191 15 %
Main.java:26			70,155 15 %
Main.java:36	<...> java.util.concurrent.TimeUnit.sleep(long)		47 0 %
Main.java:23	<...> java.lang.ClassLoader.loadClass(String)		19 0 %
Main.java:29	<...> java.util.concurrent.TimeUnit.sleep(long)		19 0 %

This time 22.04 did it with less time but just a few seconds. We can conclude that there is no significant difference between both versions of the Ubuntu OS. BUT we can see improvements when using different versions of JDK and also when adding a conditions to the iterations the table have turned, and it is not a straight answer anymore.

Conclusion.

Since on Java 17 we got a really great result on time measures, we will only specify this version on last conclusion chart.



We can easily see when adding 2 simple conditions the 3 methods are having similar running time our conclusion of this case, when you are iterating over array or a data structure and you have a lot of business logic with each loop use the Stream API, when its very simple use a regular For-Loop.

Different way to Read \ Write Files

Well read and write files is very common wherever you look, if you want to encrypt some files, you eventually will need also to read files first and after that to write them with the new encryption but this just one example. There are more than a few ways to do so, we tried to focus on our main Google results, and really hoped for to find a difference in the methods. In order to test these methods, we use 3 different ways of writing files and, then we use 2 different methods to read those files, all in one Main-Thread. Also, we compare between Ubuntu OS versions 18.04 and 22.04.

Ubuntu 18.04

Java 8

Call Tree			Time (ms)	
main	Group: 'main'		913,460	100 %
ReadWirteFiles.Main.main(String[])			913,460	100 %
Main.java:55	ReadWirteFiles.ReadWithScanner.start()		534,736	59 %
Main.java:47	ReadWirteFiles.ReadWithBufferedReader.start()		130,992	14 %
Main.java:30	ReadWirteFiles.PrintWriterMethod.start()		105,932	12 %
Main.java:38	ReadWirteFiles.FileOutputStreamMethod.start()		81,604	9 %
Main.java:21	ReadWirteFiles.BufferedWriterMethod.start()		59,168	6 %

Java 11

Call Tree			Time (ms)	
main	Group: 'main'		1,832,232	100 %
ReadWirteFiles.Main.main(String[])			1,832,232	100 %
Main.java:55	ReadWirteFiles.ReadWithScanner.start()		1,031,984	56 %
Main.java:38	ReadWirteFiles.FileOutputStreamMethod.start()		302,180	16 %
Main.java:47	ReadWirteFiles.ReadWithBufferedReader.start()		258,320	14 %
Main.java:30	ReadWirteFiles.PrintWriterMethod.start()		163,416	9 %
Main.java:21	ReadWirteFiles.BufferedWriterMethod.start()		75,540	4 %

Java 17

Call Tree			Time (ms)	
main	Group: 'main'		1,723,340	100 %
ReadWirteFiles.Main.main(String[])			1,723,340	100 %
Main.java:55	ReadWirteFiles.ReadWithScanner.start()		1,097,204	64 %
Main.java:47	ReadWirteFiles.ReadWithBufferedReader.start()		224,036	13 %
Main.java:30	ReadWirteFiles.PrintWriterMethod.start()		182,768	11 %
Main.java:38	ReadWirteFiles.FileOutputStreamMethod.start()		171,184	10 %
Main.java:21	ReadWirteFiles.BufferedWriterMethod.start()		47,544	3 %

```
Java version: 1.8
press Enter to START when profiler is READY

BufferedWriterMethod -> 1.524
PrintWriterMethod -> 2.308
FileOutputStreamMethod -> 30.562
Reading...
ReadWithBufferedReader -> 2.928
ReadWithScanner -> 19.625
BufferedWriterMethod -> 1.187
PrintWriterMethod -> 2.24
FileOutputStreamMethod -> 30.574
Reading...
ReadWithBufferedReader -> 2.861
ReadWithScanner -> 19.521
```

```
Java version: 11.0
press Enter to START when profiler is READY

BufferedWriterMethod -> 1.329
PrintWriterMethod -> 1.874
FileOutputStreamMethod -> 31.888
Reading...
ReadWithBufferedReader -> 2.163
ReadWithScanner -> 13.103
BufferedWriterMethod -> 1.203
PrintWriterMethod -> 1.827
FileOutputStreamMethod -> 31.906
Reading...
ReadWithBufferedReader -> 2.095
ReadWithScanner -> 13.087

Process finished with exit code 0
```

```
Java version: 17.0
press Enter to START when profiler is READY

BufferedWriterMethod -> 1.275
PrintWriterMethod -> 1.747
FileOutputStreamMethod -> 30.608
Reading...
ReadWithBufferedReader -> 2.163
ReadWithScanner -> 12.276
BufferedWriterMethod -> 1.181
PrintWriterMethod -> 1.698
FileOutputStreamMethod -> 30.607
Reading...
ReadWithBufferedReader -> 2.157
ReadWithScanner -> 11.985
```

Ubuntu 22.04

Java 8

Call Tree			Time (ms)	
main	Group: 'main'		916,824	100 %
ReadWirteFiles.Main.main(String[])			916,824	100 %
Main.java:55	ReadWirteFiles.ReadWithScanner.start()		533,676	58 %
Main.java:47	ReadWirteFiles.ReadWithBufferedReader.start()		131,644	14 %
Main.java:30	ReadWirteFiles.PrintWriterMethod.start()		104,224	11 %
Main.java:38	ReadWirteFiles.FileOutputStreamMethod.start()		87,968	10 %
Main.java:21	ReadWirteFiles.BufferedWriterMethod.start()		58,252	6 %

Java 11

Call Tree			Time (ms)	
main	Group: 'main'		1,584,364	100 %
ReadWirteFiles.Main.main(String[])			1,584,364	100 %
Main.java:56	ReadWirteFiles.ReadWithScanner.start()		921,332	58 %
Main.java:48	ReadWirteFiles.ReadWithBufferedReader.start()		248,036	16 %
Main.java:39	ReadWirteFiles.FileOutputStreamMethod.start()		177,712	11 %
Main.java:31	ReadWirteFiles.PrintWriterMethod.start()		161,560	10 %
Main.java:22	ReadWirteFiles.BufferedWriterMethod.start()		75,084	5 %

Java 17

Call Tree			Time (ms)	
main	Group: 'main'		1,574,764	100 %
ReadWirteFiles.Main.main(String[])			1,574,764	100 %
Main.java:55	ReadWirteFiles.ReadWithScanner.start()		1,044,204	66 %
Main.java:47	ReadWirteFiles.ReadWithBufferedReader.start()		218,800	14 %
Main.java:30	ReadWirteFiles.PrintWriterMethod.start()		176,272	11 %
Main.java:38	ReadWirteFiles.FileOutputStreamMethod.start()		90,640	6 %
Main.java:21	ReadWirteFiles.BufferedWriterMethod.start()		44,204	3 %

```
Java version: 1.8
press Enter to START when profiler is READY

BufferedWriterMethod -> 1.558
PrintWriterMethod -> 2.375
FileOutputStreamMethod -> 34.122
Reading...
ReadWithBufferedReader -> 2.853
ReadWithScanner -> 19.797
BufferedWriterMethod -> 1.246
PrintWriterMethod -> 2.266
FileOutputStreamMethod -> 34.137
Reading...
ReadWithBufferedReader -> 2.81
ReadWithScanner -> 19.59
```

```
Java version: 11.0
press Enter to START when profiler is READY

BufferedWriterMethod -> 1.417
PrintWriterMethod -> 1.846
FileOutputStreamMethod -> 35.664
Reading...
ReadWithBufferedReader -> 2.103
ReadWithScanner -> 12.89
BufferedWriterMethod -> 1.258
PrintWriterMethod -> 1.826
FileOutputStreamMethod -> 35.677
Reading...
ReadWithBufferedReader -> 2.058
ReadWithScanner -> 12.518
```

```
Java version: 17.0
press Enter to START when profiler is READY

BufferedWriterMethod -> 1.326
PrintWriterMethod -> 1.787
FileOutputStreamMethod -> 33.782
Reading...
ReadWithBufferedReader -> 2.166
ReadWithScanner -> 12.019
BufferedWriterMethod -> 1.192
PrintWriterMethod -> 1.702
FileOutputStreamMethod -> 33.659
Reading...
ReadWithBufferedReader -> 2.09
ReadWithScanner -> 11.855
```

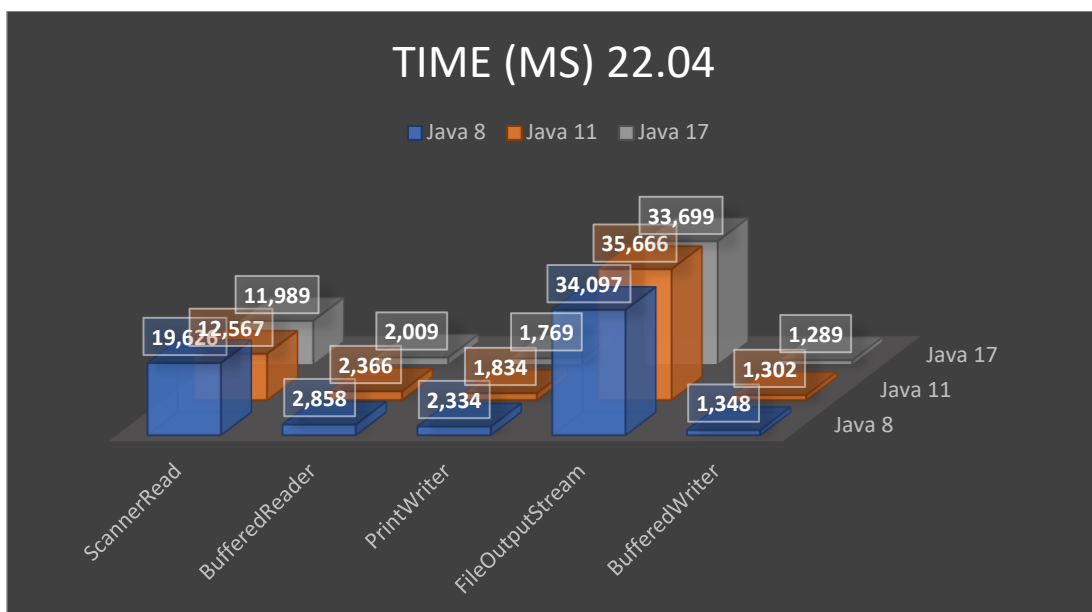
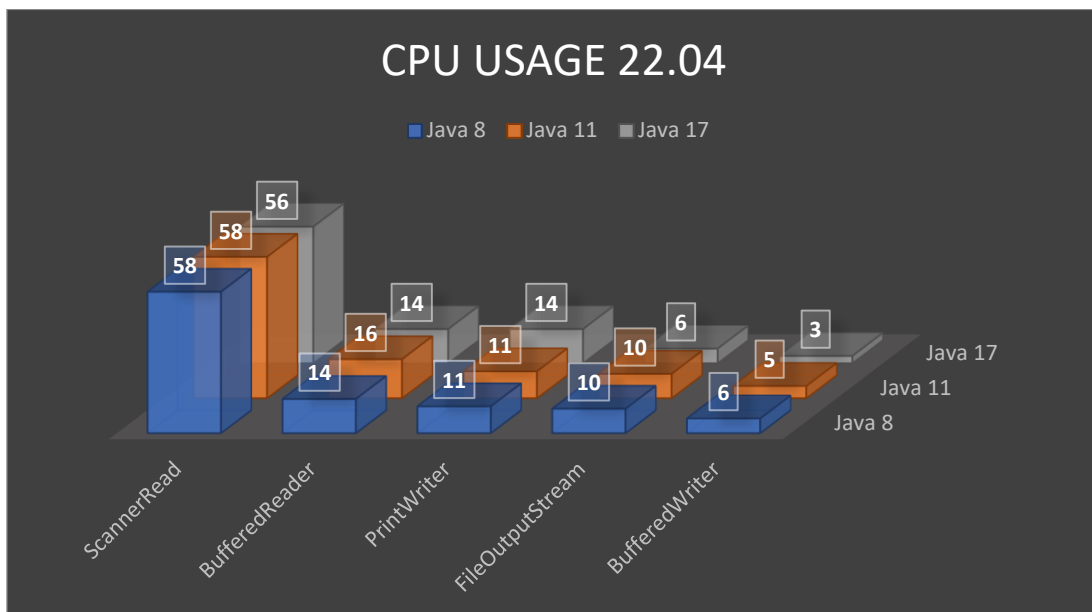
Conclusions

When looking for difference between versions of Java we did not see any significant differences in performance, but if we separate each method to see what it better be use, although there are improvements with

BufferWriter really performed in all versions the best, and also between the 2 methods that we used for reading files *BufferedReader* preform much better than the *Scanner*.

even in terms of CPU usage it is much better, not just the running time.

We did not see any differences in the OS versions of Ubuntu, but we got better results on the 22.04 when monitoring with the profiler.



Conclusions

We manage to investigate only a little amount of what Java has to offer, but we did find interesting results, first we will say there are none significant differences between Ubuntu versions, this is probably since our Java programs runs on a JVM and this is what special about Java, but we recommended if possible to upgrade to the latest version of Ubuntu (22.04) since eventually 18.04 will no longer be supported and software will need sometimes a special adjustments in order to run.

between the Java versions so far, the running time and CPU usage were more factors than the difference with functionality, since there were big improvements to the GC since Java 9, we can see that usually our programs run better, and the Heap-Memory is being used smartly.

Over the different methods to get the same results we found that – when you want to copy and array, and you want it to be more readable and less lines of code you can use the *System.arraycopy* method but a regular for-loop is also a great choice since the other methods are very CPU consuming.

With the Stream API that was hard to investigate with what we got the ‘Reduce’ terminal was very CPU consuming and when dealing with numbers it is better to converting into and *Int\Double Stream* in order to manipulate it faster and better.

Enumerations are more readable and as we found did not cost us more time or CPU usage if you familiar with using those or you want your code to be “cleaner”, you should definitely start using Enumerations.

Also iterating over a data structure, if the iterations are very simple you should use a regular for-loop to get the job done, but if the Object is complex and you have to manipulate it on every iteration like – checking with if’s and whatever is you want to do with each element you should use the Stream API, when it gets more complex your for-loop will start to perform less and less good (depending on the manipulation).

Records are also a way to make your code “cleaner” and as we found out it not consuming at all and very similar to a regular class on performance, it is new since Java 16 it is official and developers need to get used to it, in order to use it but they should since readable and clean code are very important.

Switch-case was a surprise since it really performs much better than the “if-else if” and we did not expect that, and also there are improvements with the Enhanced Switch, this is a big plus since Switches are more readable when there are a lot of conditions.

I\O – meaning read and write files there were interesting result as well, the *BufferedWriter* & *BufferedReader* were the best to use and we found a significant improvement with *Scanner* method that we did not intend to find.

So in general, since all the new features usually come to make our life better and easier as a programmer we should upgrade to stabilized version when we can and need to.

Features in each version

We will list the featured that related to our paper, the number of features for each version is different and, some are more related, and some are not.

- Java 5 - Enumerations, ForEach loop which is called enhanced ForEach loop. [\[3\]](#)
- Java 7 - String in Switch. [\[3\]](#)
- Java 8 - The famous Stream API, Lambda expressions. [\[3\]](#)
- Java 9 - Improvement in the Stream API. [\[3\]](#)
- Java 12 - Switch Expressions (Preview). [\[3\]](#)
- Java 13 - Switch Expressions enhancement. [\[3\]](#)
- Java 14 - Switch Exp` (Standard), Records (Preview), *instanceOf* pattern match (Preview). [\[3\]](#)
- Java 16 - Records (Standard), *instanceOf* pattern matching (Standard). [\[3\]](#)
- Java 17 - Pattern Matching for switch (Preview). [\[3\]](#)
- Java 18 - Pattern Matching for switch (Preview). [\[3\]](#)

As mentioned above we did not list all the features for each version, and we ignore some. There are many features which related to web development and the JVM engine, these features are out of scope for this paper.

References

- [1] <https://stackoverflow.com/questions/1906445/what-is-the-difference-between-jdk-and-jre>
- [2] <https://www.marcoBehler.com/guides/a-guide-to-java-versions-and-features>
- [3] <https://howtodoinjava.com/java-version-wise-features-history/>
- [4] <http://www.diva-portal.org/smash/record.jsf?pid=diva2%3A1443070&dswid=-2576>
- [5] <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html>